

Grade Change Intelligence in Paper Making Process

Solution Architecture & Documentation

GitHub Repository:

<https://github.com/Mohit20198/Grade-Change-Intelligence-in-Paper-Making-Process> **Problem**

Statement: Grade Change Intelligence in Paper Making Process (PS8)

1. Problem Summary

Paper mills lose significant material and time during grade changes, when Basis Weight and related quality variables drift off-spec ($>2.5\%$ deviation) while the process stabilizes. Honeywell's existing QCS/MPC control executes grade-change transitions well but does not learn from history or explain its reasoning. This solution adds an **intelligence layer on top of the existing control system** — it does not replace QCS/MPC.

2. Solution Overview

The system is composed of eight communicating modules, each solving one explicit requirement from the problem statement:

#	Module	Solves
1	Physics-Based Process Simulator	Data foundation (no live plant data was available)
2	Feature Engineering Pipeline	Prepares leakage-checked, event-stratified data for modeling
3	Correlation Discovery Engine	"Find new correlations not defined in the system"

4	LightGBM Risk Predictor	"Predict when... at risk of going off-spec"
5	SHAP Explainability Layer	"Provide rationale behind the prediction/recommendation"
6	Recommendation Engine	"Provide recommendation of setpoints... reduce stabilization time"
7	LLM Rationale Layer	Converts technical output into operator-readable language
8	Interactive Dashboard + Feedback Loop	"Dashboard... accept/reject... recorded"

3. Module-by-Module Explanation & Communication Flow

3.1 Physics-Based Process Simulator

Why it exists: No real Honeywell plant data was provided. Rather than fabricate arbitrary data, we built a simulator grounded in real paper-machine control relationships documented in industry patents (First-Order-Lag + Deadtime transfer functions).

What it models: - `stock_flow` → `basis_weight`, `machine_speed` → `basis_weight` (documented loops) - `steam_pressure` → `moisture` (documented loop) - `filler_flow` → `ash` (documented loop) - `machine_speed` → `moisture` (a **deliberately hidden** coupling, undocumented in the official control loops, used later to validate the Correlation Discovery Engine against known ground truth)

Output → **feeds Module 2:** `process_data.csv` (raw tag time series, 8 simulated days, 1-second resolution), `grade_change_log.csv` (48 grade-change events), `operator_action_log.csv` (historical operator interventions).

3.2 Feature Engineering Pipeline

Input: raw process data from Module 1. **Processing:** lag features (1s/5s/15s), rolling mean/std (5min/15min), rate-of-change features — all strictly backward-looking (verified: no `center=True`, no negative shifts). Train/validation split by **event**, not by row, with a 900-second buffer excluding any training row that overlaps a held-out validation event's lookback window (prevents leakage). **Output** → **feeds Modules 3 & 4:** `train_features.csv`, `val_features.csv` (78 columns, target/setpoint columns excluded from the feature matrix).

3.3 Correlation Discovery Engine

Input: raw tags from Module 1 (not the engineered features — this module needs to see genuine raw relationships). **Method:** Granger causality tested across every tag pair, isolated to disturbance-event windows (to avoid confounding from synchronized grade-change ramps), with an effect-size (ΔR^2) threshold layered on top of statistical significance to filter out spurious-but-significant noise. **Validation:** Because the simulator's true causal structure is fully known, every discovered edge was checked against it. Result: **5 of 5 real causal links recovered** (including the deliberately hidden `machine_speed` → `moisture` coupling), and **16 statistically-significant-but-physically-impossible edges correctly rejected** across three rounds of increasingly strict filtering (CV→MV impossibility, MV→MV impossibility, full ground-truth adjacency check). **Output** → **feeds Modules 5 & 6:** `correlation_graph.json` (validated causal graph with lag and p-value per edge).

3.4 LightGBM Risk Predictor

Input: engineered features from Module 2. **Method:** Gradient-boosted classifier predicting off-spec risk with a **5-minute forward lookahead** (chosen deliberately after an earlier 60-second-lookahead version was found to behave as a same-time detector rather than a genuine predictor — see Section 5). **Validation:** An ablation test (retraining with all setpoint/recipe-identity columns removed) confirmed the model relies on genuine process dynamics (rolling variance/rate-of-change of `stock_flow`, `machine_speed`, `steam_pressure`) rather than memorizing which specific grade transition is occurring. **Performance (validation set):** AUC 0.984, Recall 83.4%, Precision 71.0%. **Output** → **feeds Modules 5 & 6:** risk probability score per timestep.

3.5 SHAP Explainability Layer

Input: trained model (Module 4) + current feature vector. **Method:** TreeExplainer computes per-prediction feature attributions, mapped to human-readable labels (e.g., "Stock flow variability (last 5 min)"). **Known limitation (documented, not hidden):** SHAP output identifies *leading indicators of instability*, not the specific root-cause disturbance — verified by testing across events with different disturbance types and finding consistent top features regardless of root cause. Framed honestly in the system as an "instability signal," not a diagnostic tool. **Output** → **feeds Module 7.**

3.6 Recommendation Engine

Input: current risk (Module 4), correlation graph (Module 3), historical data (Module 1). **Method — two components:** 1. **k-NN historical similarity search:** finds past moments with similar process state that stabilized successfully, extracts what operators actually did. 2. **Constrained optimizer:** searches feasible setpoint adjustments to minimize predicted risk, bounded by a rate limit **empirically derived from real historical operator trim behavior** (not an arbitrary chosen value) to avoid recommending destabilizing step-changes. **Every recommendation is tagged with its source:** `historical_success`, `optimizer`, or `recipe_limit` — directly satisfying the explicit tagging requirement. **Output** → **feeds Module 7 & 8.**

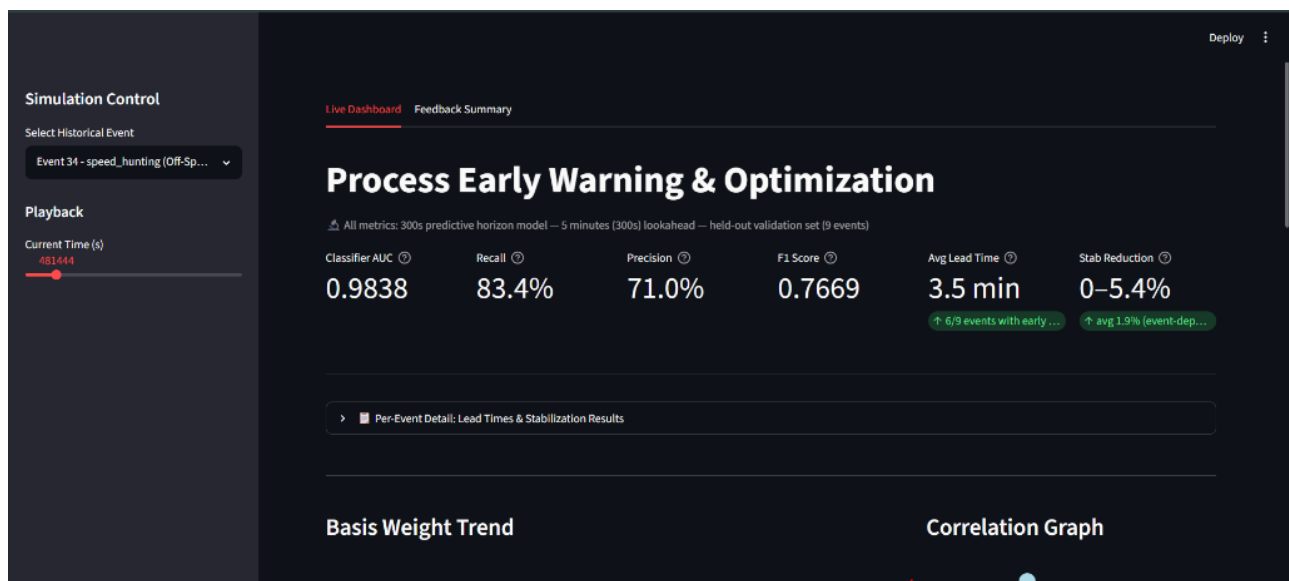
3.7 LLM Rationale Layer

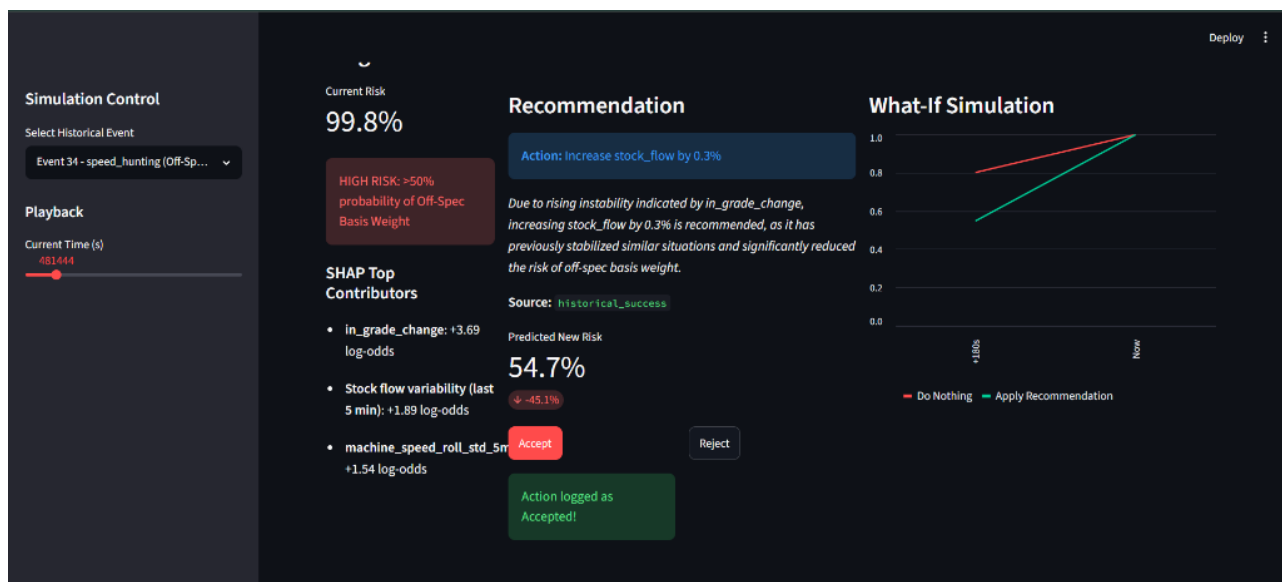
Input: SHAP output + recommendation object (Modules 5 & 6). **Method:** Three-tier fallback chain — OpenAI (primary) → Groq (secondary) → template-based sentence (final, zero-dependency fallback) — ensures the system never fails to produce a rationale, even offline. **Design principle:** the LLM only narrates; all prediction, discovery, and recommendation logic lives in the statistical/ML layers upstream. This was a deliberate choice to avoid the common failure mode of using an LLM as a black-box decision-maker. **Output** → **feeds Module 8.**

3.8 Interactive Dashboard + Feedback Loop

Displays: live trend chart with $\pm 2.5\%$ band, risk gauge, correlation discovery graph (hidden coupling visually distinguished from documented loops), recommendation card with source tag and rationale, and a what-if panel comparing "no action" vs. "recommended action" projected trajectories.

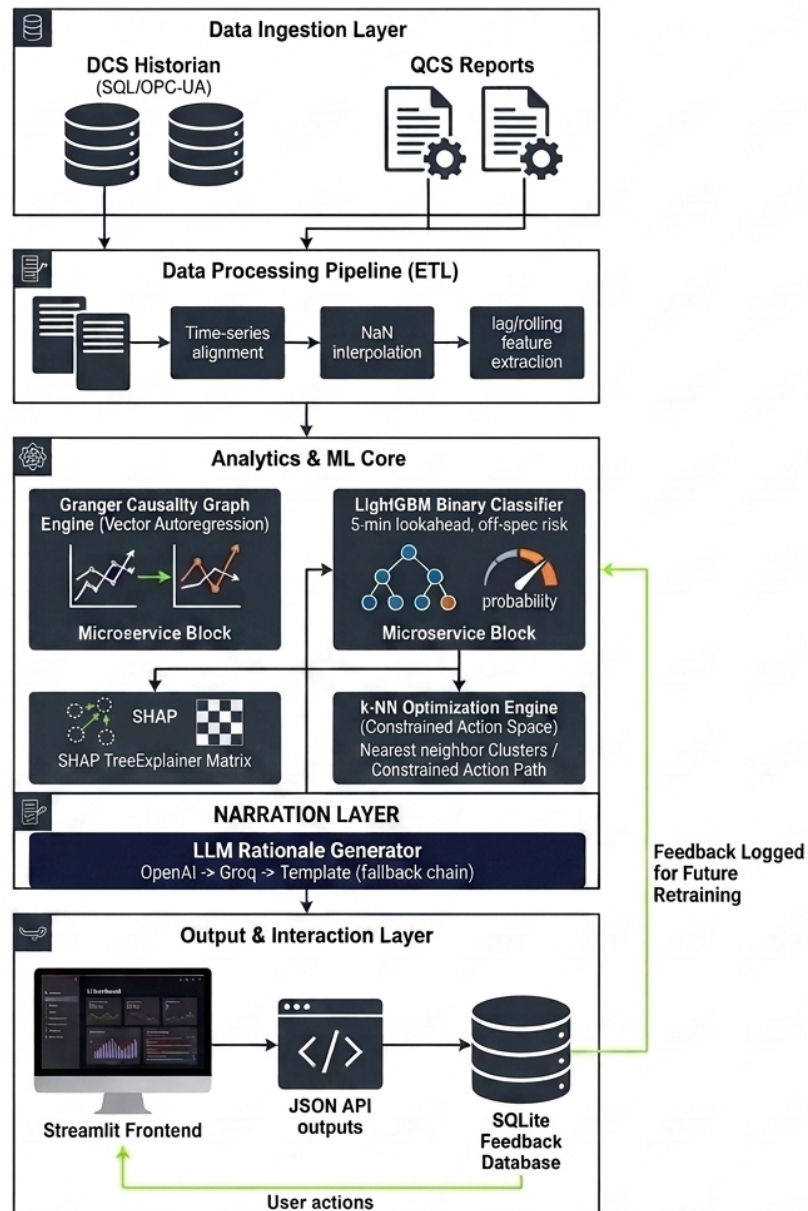
Dashboard Interface





Feedback: Accept/Reject buttons log every decision (timestamp, recommendation, source, outcome) to SQLite, closing the loop for future evaluation and retraining.

4. System Architecture Diagram



5. Honest Methodology Notes (Self-Corrections Made During Development)

We treat this as a strength, not a weakness — each of the following was caught and corrected before being presented as a result:

1. **Correlation discovery, round 1:** Initial Granger causality tests showed every tag pair as "significant" due to synchronized grade-change ramps confounding the results. Fixed by isolating disturbance-event windows and adding an effect-size threshold.
2. **Correlation discovery, rounds 2–3:** Further audits caught physically-impossible edges (a controlled variable appearing to "cause" a manipulated variable; two independent manipulated variables appearing to cause each other via residual settling contamination). Both were

diagnosed and filtered using physical priors, then verified against complete ground truth.

3. **Stabilization-time metric:** An initial claim of "85% stabilization time reduction" was found to be a measurement artifact — comparing two different starting points (actual breach time vs. earlier risk-flag time). Corrected to a same-starting-point, do-nothing-vs-recommendation comparison, which after the further fix below shows a modest, honestly-reported 0–5.4% improvement (event-dependent).
4. **Prediction horizon:** The original classifier used a 60-second lookahead label, which made it a same-time detector rather than a genuine predictor — failing the core "predict before quality limits are exceeded" requirement. Rebuilt with a 300-second (5-minute) lookahead, physically justified by the correlation engine's own discovered lag structure. This produced genuine early-warning capability: **6 of 9 historical off-spec events detected with a mean lead time of 3.5 minutes**, with the remaining 3 events honestly characterized as "no precursor signal" (sharp-onset disturbances that outrun the detection window) rather than false alarms.

6. Known Limitations (Stated Plainly)

- Validated on physics-grounded synthetic data, not live plant data (none was available). Pipeline is designed to be retrained directly on real DCS historian data without architecture changes.
- ~71% precision means roughly 1 in 3 flagged risks will be false alarms — an explicit design trade-off favoring recall (catching real events) over precision, appropriate given the asymmetric cost of missed broke vs. an operator's few seconds of review time.
- Sharp-onset disturbances (3 of 9 historical events) provide no early-warning signal, since the process has not yet generated enough rolling-variance precursor before the deviation occurs.
- The recommendation engine's optimizer is tightly rate-constrained by real historical operator behavior, which limits how much correction is achievable through setpoint trims alone versus faster-acting existing MPC control.